SQL & DBMS Practical Work

Name: Jajula Gnana Deepak **ID:**2300030263
Subject: Database Management System (DBMS)
Tools Used: MySQL Workbench

Task 1 — Create College Database and Students Table

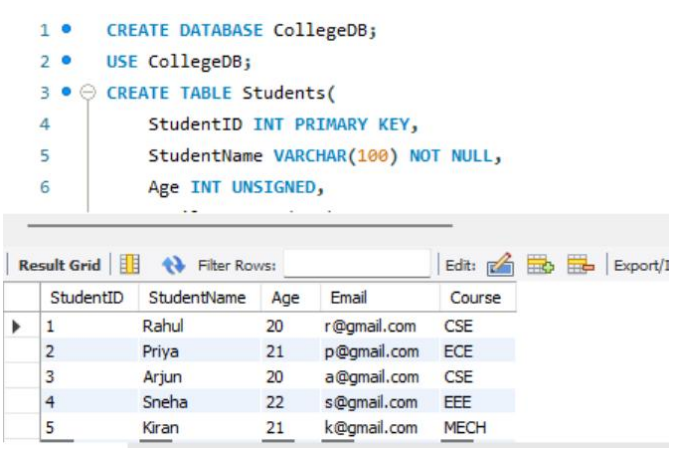
Objective

Create a database, create a Students table, insert 5 student records, and display the records.

```
CREATE DATABASE CollegeDB;

USE CollegeDB;


CREATE TABLE Students(
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(100) NOT NULL,
   Age INT UNSIGNED,
   Email VARCHAR(100),
   Course VARCHAR(100)
);
INSERT INTO Students VALUES
(1,'Rahul',20,'r@gmail.com','CSE'),
(2,'Priya',21,'p@gmail.com','ECE'),
(3,'Arjun',20,'a@gmail.com','CSE'),
(4,'Sneha',22,'s@gmail.com','EEE'),
(5,'Kiran',21,'k@gmail.com','MECH');
SELECT * FROM Students;
```



What I Did

- Created the CollegeDB database.
- Selected the database using USE.
- Created the Students table.
- Applied PRIMARY KEY, NOT NULL, and UNSIGNED constraints.
- Inserted 5 student records.
- Displayed all student records using SELECT.

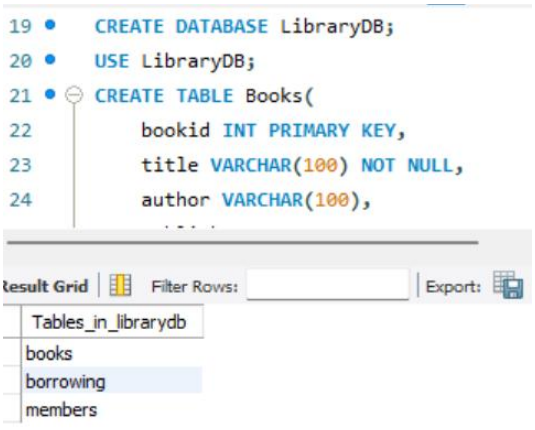
Task 2 — Create Library Database

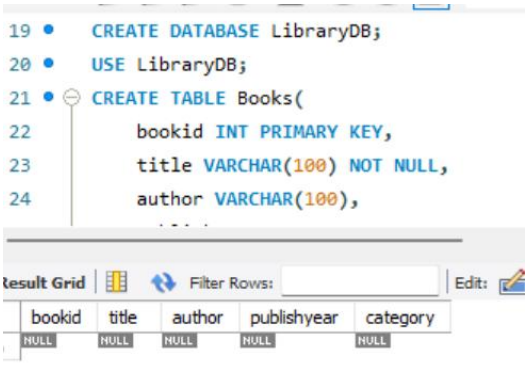
Objective

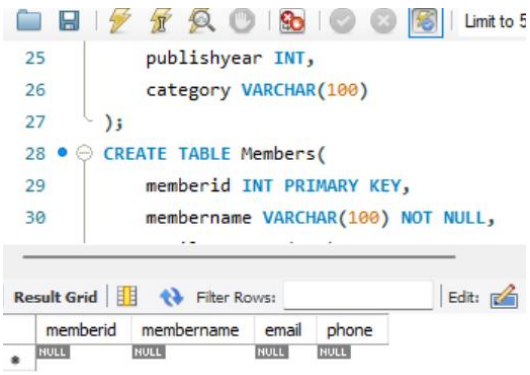
Create a library database containing Books, Members, and Borrowing tables and establish relationships using foreign keys.

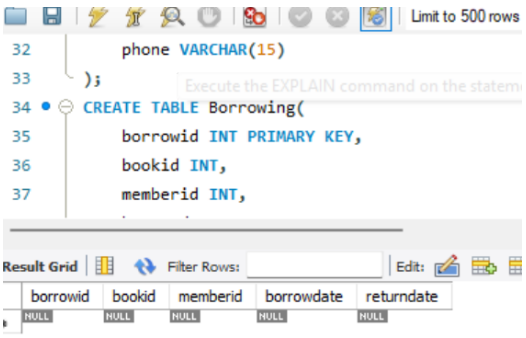
```
CREATE DATABASE LibraryDB;
USE LibraryDB;
CREATE TABLE Books(
   bookid INT PRIMARY KEY,
   title VARCHAR(100) NOT NULL,
   author VARCHAR(100),
   publishyear INT,
   category VARCHAR(100)
);
CREATE TABLE Members(
   memberid INT PRIMARY KEY,
   membername VARCHAR(100) NOT NULL,
   email VARCHAR(100),
   phone VARCHAR(15)
);
CREATE TABLE Borrowing(
   borrowid INT PRIMARY KEY,
   bookid INT,
```

```
    memberid INT,
    borrowdate DATE,
    returndate DATE,
    FOREIGN KEY (bookid) REFERENCES Books(bookid),
    FOREIGN KEY (memberid) REFERENCES Members(memberid)
);
SHOW TABLES;
SELECT * FROM Books;
SELECT * FROM Members;
SELECT * FROM Borrowing;
```









What I Did

- Created the LibraryDB database.
- Created the Books table to store book details.
- Created the Members table to store member details.
- Created the Borrowing table to store borrowing transactions.
- Used PRIMARY KEY for unique identification.
- Used FOREIGN KEY to establish relationships between tables.
- Used SHOW TABLES to display the tables.
- Used SELECT to view the table contents.

Task 3 — Create Hospital Database

Objective

Create a HospitalDB database, create a Patients table, insert patient records, and display them.

```
CREATE DATABASE HospitalDB;

USE HospitalDB;


CREATE TABLE Patients(
   patientid INT PRIMARY KEY,
   patientname VARCHAR(100) NOT NULL,
   age INT UNSIGNED,
   gender VARCHAR(15),
   healthissue VARCHAR(100)
);


INSERT INTO Patients VALUES
(1,'Rahul',25,'M','Fever'),
```

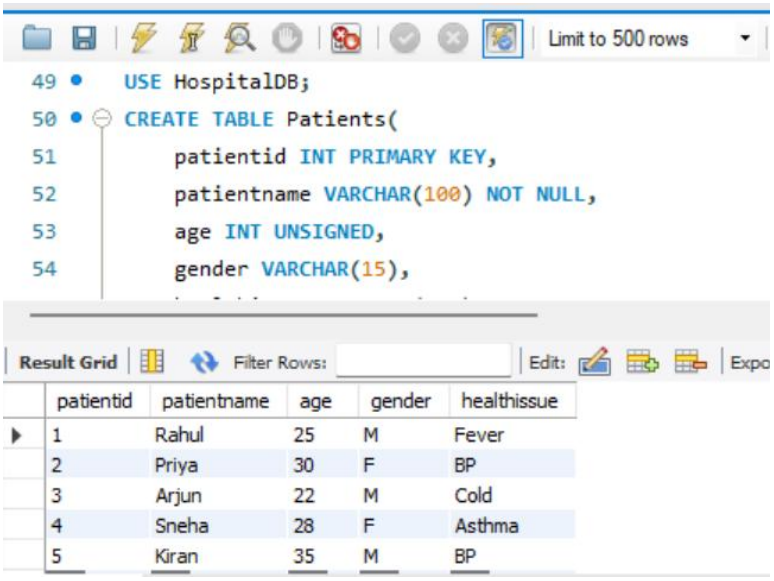
(2,'Priya',30,'F','BP'),

(3,'Arjun',22,'M','Cold'),

(4,'Sneha',28,'F','Asthma'),

(5,'Kiran',35,'M','BP');

SELECT * FROM Patients;



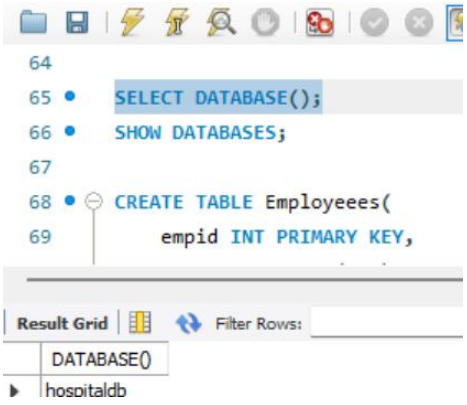
patientid	patientname	age	gender	healthissue
1	Rahul	25	M	Fever
2	Priya	30	F	BP
3	Arjun	22	M	Cold
4	Sneha	28	F	Asthma
5	Kiran	35	M	BP

Task 4 — Check Current Database

Objective

Find the currently selected database.

SELECT DATABASE();



What I Did

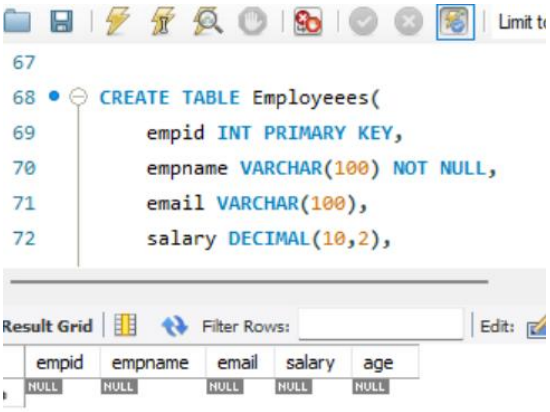
- Used SELECT DATABASE() to identify the currently selected database.

Task 5 — Create Employee Table

Objective

Create an employee table with appropriate data types and constraints.

```
CREATE TABLE Employeees(
   empid INT PRIMARY KEY,
   empname VARCHAR(100) NOT NULL,
   email VARCHAR(100),
   salary DECIMAL(10,2),
   age INT UNSIGNED
);
SELECT * FROM Employeees;
```



What I Did

- Created the Employeees table.
- Used PRIMARY KEY for employee identification.
- Used NOT NULL for employee name.
- Used DECIMAL(10,2) for salary.
- Used UNSIGNED for age.
- Displayed the table contents.

Task 6 — College Student Management System

Objective

Create a student management database with constraints and perform filtering and data-integrity tests.

```
CREATE DATABASE CollegeManagement;
USE CollegeManagement;
```

```
CREATE TABLE Students(
   studentid INT UNSIGNED PRIMARY KEY,
   studentname VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   phone VARCHAR(15) UNIQUE,
   age INT UNSIGNED NOT NULL,
   dept VARCHAR(50) NOT NULL,
   year INT UNSIGNED NOT NULL,
   cgpa DECIMAL(3,2) UNSIGNED
);
INSERT INTO Students VALUES
(1,'Rahul Sharma','rahul@gmail.com','9876543210', 20, 'CSE', 2, 8.50),
(2,'Priya Reddy','priya@gmail.com','9876543211', 21, 'ECE', 3, 9.10),
(3,'Arjun Kumar','arjun@gmail.com','9876543212', 19, 'CSE', 1, 7.80),
(4,'Sneha Rao','sneha@gmail.com','9876543213', 22, 'EEE', 4, 8.90),
(5,'Kiran Patel','kiran@gmail.com','9876543214', 21, 'MECH', 3, 7.50),
(6,'Ananya Das','ananya@gmail.com','9876543215', 20, 'CSE', 2, 9.20),
(7,'Vikram Singh','vikram@gmail.com','9876543216', 23, 'CIVIL', 4, 8.10),
(8,'Neha Gupta','neha@gmail.com','9876543217', 19, 'ECE',1,8.70),
(9,'Rohan Verma','rohan@gmail.com','9876543218', 22, 'CSE',4, 9.00),
(10,'Meena Joshi','meena@gmail.com','9876543219',20,'IT',2,7.90);

SELECT * FROM Students;

SELECT *
FROM Students
WHERE dept='CSE';

SELECT *
FROM Students
WHERE cgpa>8;
```

```
SELECT *

FROM Students

WHERE age>20;


INSERT INTO Students VALUES

(1, 'T', 't@gmail.com', '9876500000', 20, 'CSE', 2, 8.00);


INSERT INTO Students VALUES

(11, 'Test Student', 'rahul@gmail.com', '9876500011', 21, 'CSE', 3, 8.20);


INSERT INTO Students VALUES

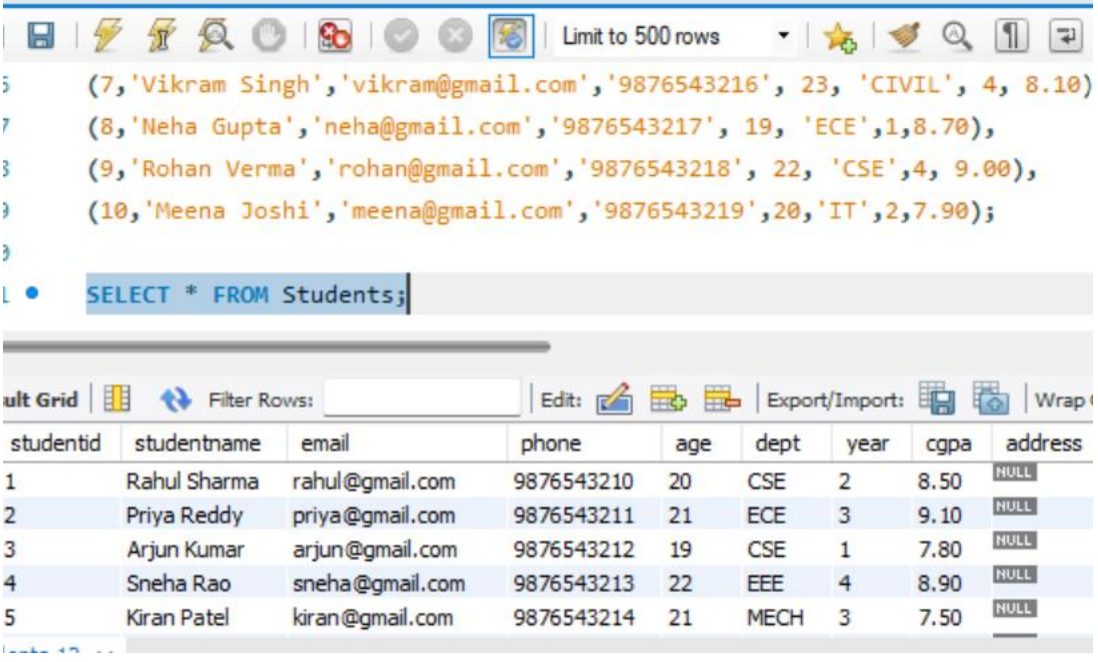
(12, NULL, 'test12@gmail.com', '9876500012', 20, 'CSE', 2, 8.00);


INSERT INTO Students VALUES

(13, 'Test Student', 'test13@gmail.com', '9876500013', -20, 'CSE', 2, 8.00);


ALTER TABLE Students

ADD address VARCHAR(200);
```



What I Did

- Displayed students from the **CSE department** using the WHERE clause.
- Displayed students with **CGPA greater than 8**.

- Displayed students whose **age is above 20**.
- Tested the **PRIMARY KEY** constraint by inserting a duplicate Student ID.
- Tested the **UNIQUE** constraint by inserting a duplicate Email.
- Tested the **NOT NULL** constraint by inserting a NULL student name.
- Tested the **UNSIGNED** constraint by inserting a negative age.
- Modified the Students table by adding a new **Address** column using ALTER TABLE.

Task 7 — Normalization

Objective

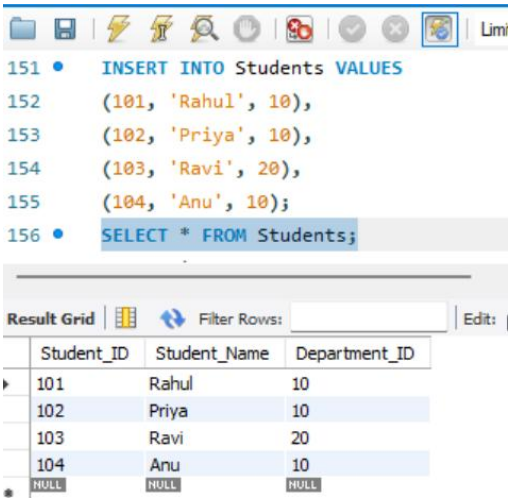
Separate student and department information into related tables to reduce data redundancy.

```
CREATE DATABASE StudentDB;

USE StudentDB;


CREATE TABLE Departments(
   Department_ID INT PRIMARY KEY,
   Department VARCHAR(50),
   HOD VARCHAR(100)
);


CREATE TABLE Students(
   Student_ID INT PRIMARY KEY,
   Student_Name VARCHAR(100) NOT NULL,
   Department_ID INT,
   FOREIGN KEY (Department_ID)
      REFERENCES Departments(Department_ID)
);
```



```
INSERT INTO Students VALUES
(101, 'Rahul', 10),
(102, 'Priya', 10),
(103, 'Ravi', 20),
(104, 'Anu', 10);
SELECT * FROM Students;
```

Result Grid | Filter Rows: | Edit:

Student_ID	Student_Name	Department_ID
101	Rahul	10
102	Priya	10
103	Ravi	20
104	Anu	10
NULL	NULL	NULL

What I Did

- Created separate Students and Departments tables.
- Used Department_ID to connect the two tables.
- Added a foreign key relationship.
- Reduced repeated department and HOD information.

Insert Normalized Data

INSERT INTO Departments VALUES

(10, 'CSE', 'Dr. Kumar'),

(20, 'ECE', 'Dr. Sharma');

INSERT INTO Students VALUES

(101, 'Rahul', 10),

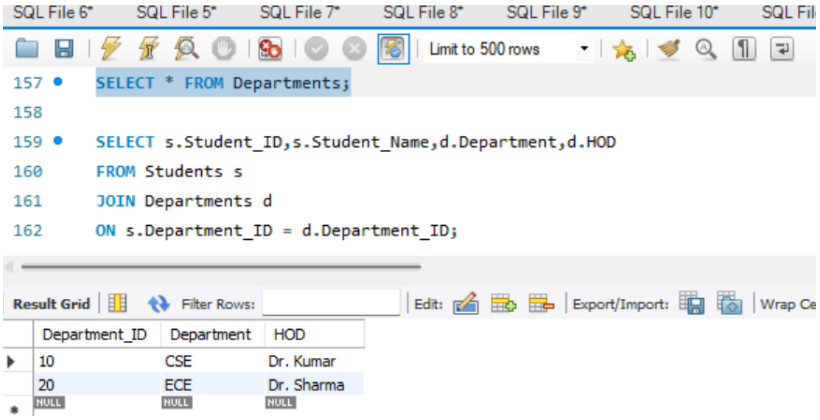
(102, 'Priya', 10),

(103, 'Ravi', 20),

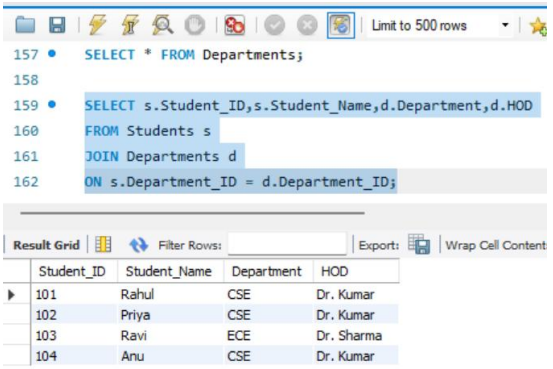
(104, 'Anu', 10);

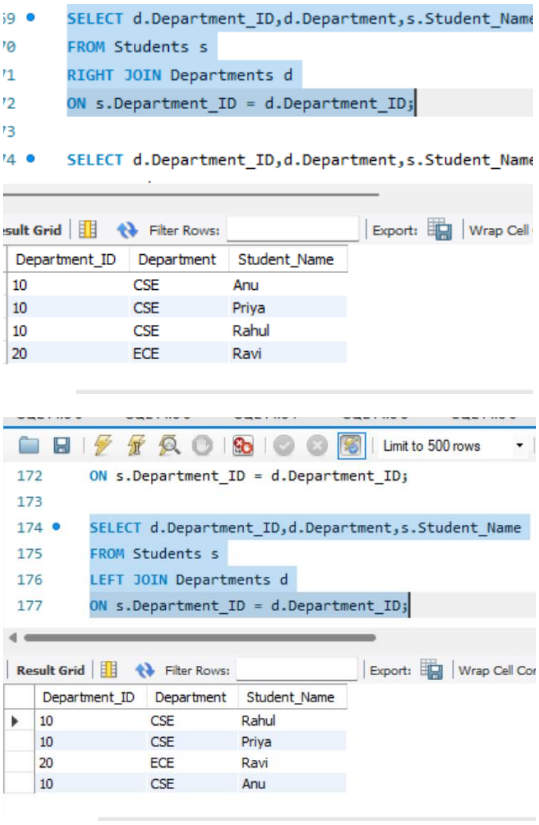
What I Did

Inserted department information separately and connected students to departments using Department_ID.



Task 8 — INNER JOIN,LEFT JOIN, RIGHT JOIN





Task 9 — FULL OUTER JOIN Using UNION

Objective

Display all matching and non-matching records from both tables.

SELECT s.Student_ID,s.Student_Name,d.Department

FROM Students s

LEFT JOIN Departments d

ON s.Department_ID = d.Department_ID

UNION

SELECT s.Student_ID,s.Student_Name,d.Department

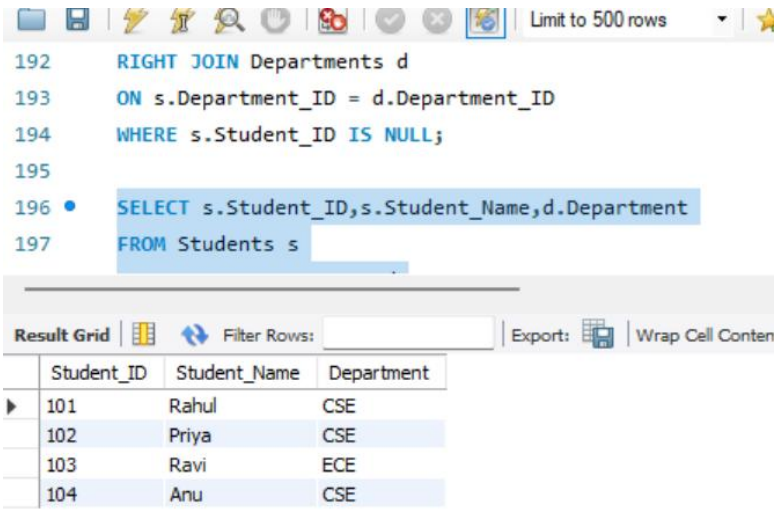
FROM Students s

RIGHT JOIN Departments d

ON s.Department_ID = d.Department_ID;

What I Did

- Used LEFT JOIN to get all students.
- Used RIGHT JOIN to get all departments.
- Used UNION to combine both results.
- Implemented the concept of a FULL OUTER JOIN in MySQL.



```
192     RIGHT JOIN Departments d
193     ON s.Department_ID = d.Department_ID
194     WHERE s.Student_ID IS NULL;
195
196     SELECT s.Student_ID,s.Student_Name,d.Department
197     FROM Students s
```

Limit to 500 rows

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

Student_ID	Student_Name	Department
101	Rahul	CSE
102	Priya	CSE
103	Ravi	ECE
104	Anu	CSE